

프로젝트 제목 : 계명크라프트

과 목 : 2D 게임프로그래밍

담당교수 : 조현철

학 과 : 게임소프트웨어학과

학 번 : 5820721

이 름 : 최승민

깃 허 브 :

https://github.com/stonpass/2d_game_programming_final.git

시연영상 : <https://youtu.be/bNpzt9BeKHs>

[목차]

1. 프로젝트 개요	3.
1) 작품주제 선정 배경	3.
2) 벤치마킹 자료	4.
3) 개발 환경 설명	5.
2. 개발 프로그램	6.
1) 사용되는 파일의 구성도	6.
2) UI/UX 설계	10.
3. 테스트 및 검증	13.
1) 테스트 및 검증	13.
2) 문제 해결 과정	15.
4. 성과 및 결론	17.
1) 기대효과 및 활용방안	17.
2) 프로젝트 성과	17.
3) 후기	18.
5. 참고문헌	19.
1) 참고자료	19.

1. 프로젝트 개요

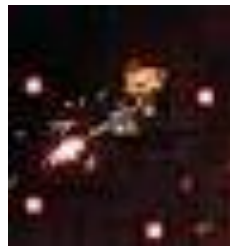
● 작품주제 선정 배경

본 작품의 주제는 RTS(실시간 전략 시뮬레이션) 게임인 '스타크래프트'를 고도(Godot) 엔진으로 구현하는 것이다.

주제를 선정하게 된 계기는 학기 시작 전 개인적으로 게임 개발을 공부하면서 유니티 엔진을 활용하여 2D 스프라이트 캐릭터의 이동 시스템을 구현한 경험에서 시작되었다. 당시 게임 '엔터 더 건전(Enter the Gungeon)'의 캐릭터 이동 방식을 참고하여 구현하였는데, 총을 들고 이동하는 2D 캐릭터를 개발하는 과정에서 자연스럽게 스타크래프트의 유닛들이 떠올랐다.

이후 2D 게임프로그래밍 과목의 기말 프로젝트 주제를 고민하던 중, 단순한 캐릭터 이동 구현을 넘어 다수의 유닛을 제어하고 생산하는 시스템을 만들어 보고 싶다는 생각이 들었다. 또한 학교 내 행사박물관을 게임 내 핵심 건물로 설정하여, 유닛들이 자원을 채취하고 이를 저장하는 과정을 구현하면 흥미로운 프로젝트가 될 것이라 판단하였다.

따라서 본 프로젝트에서는 스타크래프트의 핵심 요소인 유닛 생산, 자원 채취, 군중 이동 및 경로 탐색 시스템을 구현하는 것을 목표로 하였으며, 이를 통해 실시간 전략 게임의 구조를 이해하고 다수의 객체를 효율적으로 제어하는 기술을 학습하고자 본 주제를 선정하였다.



『게임 '엔터 더 건전(Enter the Gungeon)'의 플레이 이미지』



『개인적으로 공부한 2D 캐릭터의 이동 방식 학습 경험』

● 벤치마킹 자료

본 프로젝트는 RTS(실시간 전략 시뮬레이션) 게임인 스타크래프트를 주요 벤치마킹 대상으로 삼아 개발하였다. 게임의 전체적인 UI 구성, 유닛 선택 방식, 명령 패널, 미니맵, 효과음 사용 방식 및 조작체계 등을 참고하여 RTS 장르의 직관적인 플레이 경험을 구현하고자 하였다.



또한 인게임의 최종 목표는 적의 핵심 건물인 '저주받은 청금석'을 파괴하는 것으로 설정하였다. 이러한 목표는 스타크래프트의 유즈맵인 '신전 부수기'에서 영감을 얻었다. '신전 부수기'는 적 진영의 특정 건물을 파괴하면 승리하는 방식의 유즈맵으로, 본 프로젝트 역시 이를 응용하여 플레이어가 유닛을 생산하고 자원을 관리하며 최종적으로 적의 핵심 건물을 파괴하는 것을 목표로 설계하였다.



『스타크래프트 유즈맵 '신전 부수기' 플레이 화면』

이를 통해 스타크래프트의 전략적 재미와 유즈맵의 명확한 목표 구조를 결합하여 재미있게 플레이할 수 있는 RTS 게임을 구현하고자 하였다.

● 개발 환경 설명

o HW 구성

- 개발 장비: Windows 기반 PC
- 입력 장치: 키보드 및 마우스
- 실행 환경: 2D 그래픽 실행이 가능한 일반 데스크탑/노트북 환경

o SW 구성

- 개발 엔진: Godot Engine 4.6
- 개발 언어: GDScript
- 프로젝트 형식: 2D RTS(실시간 전략 시뮬레이션) 게임
- 운영체제: Windows
- 렌더링 방식: Godot 2D 렌더링 기반

o 개발 도구 및 IDE

- Godot Editor: 씬 구성, 노드 배치, 스크립트 작성 및 실행
- GDScript Editor: Godot 내장 스크립트 편집기 사용
- 파일 구성: .tscn 씬 파일과 .gd 스크립트 파일을 분리하여 관리

o 기타 사용한 라이브러리/플랫폼/서버

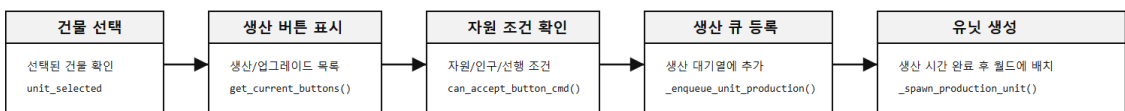
- NavigationAgent2D: 유닛의 경로 탐색 및 이동 처리
- NavigationRegion2D: 유닛이 이동 가능한 맵 영역 구성
- TileMapLayer: 타일 기반 맵 구성 및 건설 가능 영역 판단
- Godot UI 시스템: HUD, 정보창, 명령 패널, 미니맵 구성
- 별도의 외부 서버는 사용하지 않음

o 기능 설계도

1) 유닛 제어 흐름



2) 건물 제어 및 유닛 생산 흐름

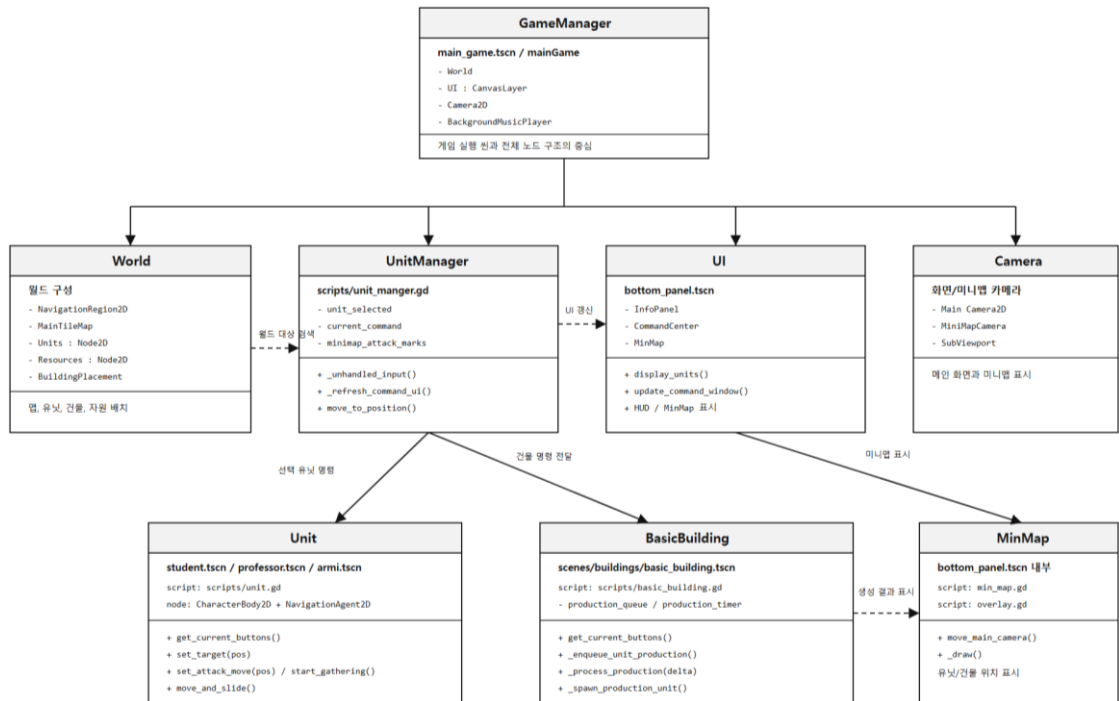


3) UI 정보 갱신 흐름



2. 개발 프로그램 설명

● 사용되는 파일의 구성도



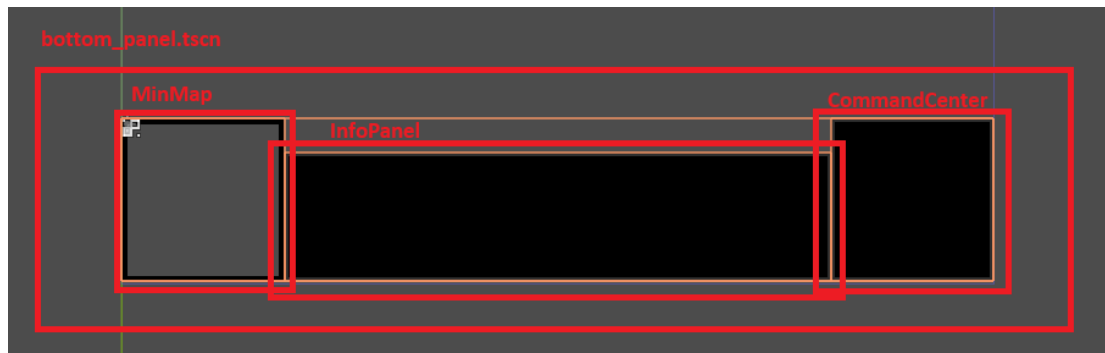
본 다이어그램은 전체 시스템 구성을 GameManager를 중심으로 정리한 구조도이다. GameManager는 main_game.tscn의 중심 노드로, 게임 실행에 필요한 주요 하위 요소들을 연결하고 관리하는 역할을 한다.

World/

Background	# 맵 배경 이미지 표시
NavigationRegion2D	# 유닛 이동 가능 영역 관리
MainTileMap	# TileMapLayer 기반 맵 타일 구성
	# 지형, 장애물, 건설 가능 영역 판단에 사용
Units	# 현재 월드에 배치된 유닛 컨테이너
Student	# 초기 배치된 학생 유닛
.....	
Resources	# 자원 및 유적 오브젝트 컨테이너
ResourceDeposit...	# 채집 가능한 자원 오브젝트
RuinsSite...	# 유적/특수 자원 지점
BuildingPlacement	# 건물 배치 모드 및 건설 가능 여부 처리
UnitTileOccupancy	# 유닛의 타일 점유 상태 관리
Camera2D	# 메인 게임 화면 카메라

World 는 실제 게임이 진행되는 공간으로, NavigationRegion2D, MainTileMap, Units, Resources, BuildingPlacement 등의 노드를 포함한다. 이를 통해 맵 지형, 유닛 배치, 자원 배치, 건물 배치 기능이 구성된다. 유닛과 건물은 World 안에 배치되며, 플레이어의 명령에 따라 이동, 생산, 채집, 건설 등의 기능을 수행한다.

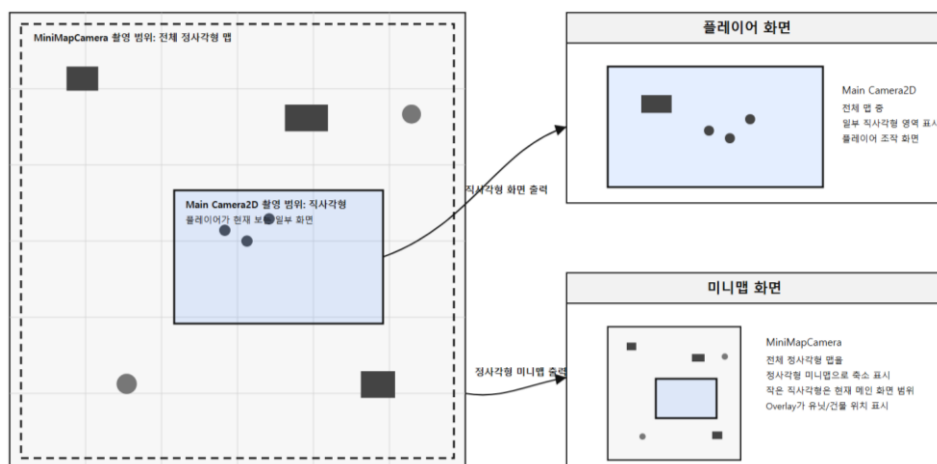
UnitManager 는 플레이어의 입력을 처리하고 선택된 유닛 또는 건물을 관리하는 핵심 스크립트이다. 마우스 클릭이나 드래그를 통해 선택된 대상은 unit_selected 목록에 저장되며, 이후 이동 명령, 건물 명령, UI 갱신 요청이 UnitManager 를 통해 전달된다. 즉 UnitManager 는 플레이어 입력과 게임 오브젝트 사이를 연결하는 중간 관리자 역할을 한다.



『UI 인 bottom_panel.tscn 의 씬 화면』

UI 는 bottom_panel.tscn 을 기반으로 구성되며, InfoPanel, CommandCenter, MinMap 으로 나뉜다. InfoPanel 은 선택된 유닛이나 건물의 정보를 표시하고, CommandCenter 는 선택 대상이 수행할 수 있는 명령 버튼을 표시한다. MinMap 은 전체 맵의 축소 화면과 유닛 및 건물 위치를 표시하여 플레이어가 전장 상황을 빠르게 파악할 수 있도록 한다.

Camera 는 메인 화면을 보여주는 Main Camera2D 와 미니맵 표시를 위한 MiniMapCamera 로 나뉜다. 메인 카메라는 플레이어가 실제로 조작하는 화면을 담당하고, MiniMapCamera 는 SubViewport 를 통해 전체 맵을 축소하여 미니맵 영역에 표시한다.

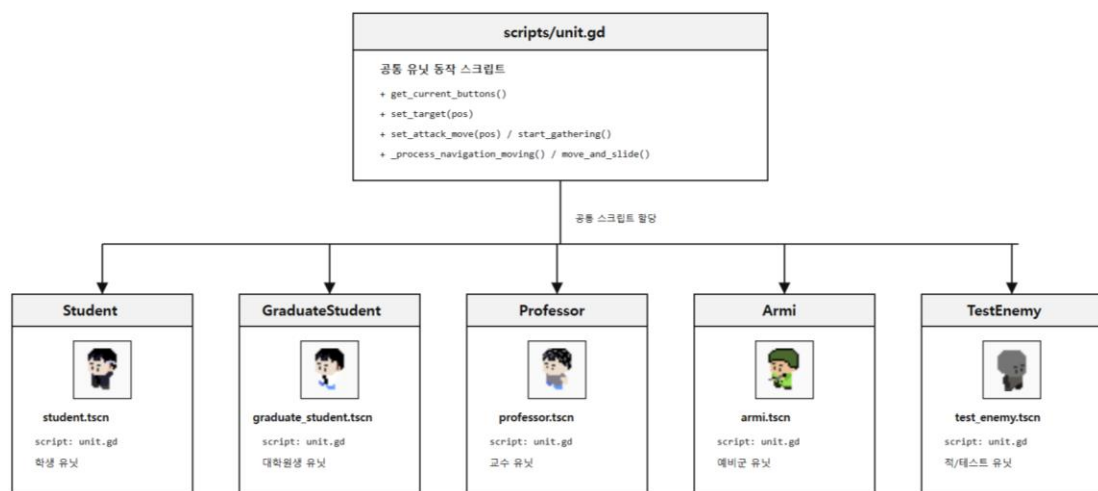


『전체맵에서 메인카메라와 미니맵카메라의 차이점』

Unit 은 student.tscn, professor.tscn, armi.tscn 등의 유닛 씬으로 구성되며, unit.gd 스크립트를 통해 이동, 공격, 채집 등의 동작을 수행한다. 유닛은 NavigationAgent2D 를 이용하여 목적지까지의 경로를 계산하고, move_and_slide()를 통해 실제 이동을 처리한다.

유닛은 각 유닛별로 별도의 씬을 가지지만, 이동, 공격, 채집, 명령 처리 등의 공통 동작은 unit.gd 스크립트에서 처리한다. 유닛의 종류는 각 씬에 설정된 이름, 이미지, 능력치 등의 값으로 구분된다.

적 유닛 역시 별도의 전용 스크립트를 사용하는 것이 아니라, 아군 유닛과 동일한 unit.gd 스크립트를 사용한다. 다만 test_enemy.tscn 은 Enemy 그룹에 속하도록 설정되어 있어 아군 유닛과 구분된다. 실제 게임 진행 중 등장하는 적 유닛은 game_result_manager.gd 에서 test_enemy.tscn 을 생성하는 방식으로 구현되어 있으며, 생성된 적 유닛은 unit.gd 의 이동, 공격, 탐지 로직을 기반으로 동작한다.

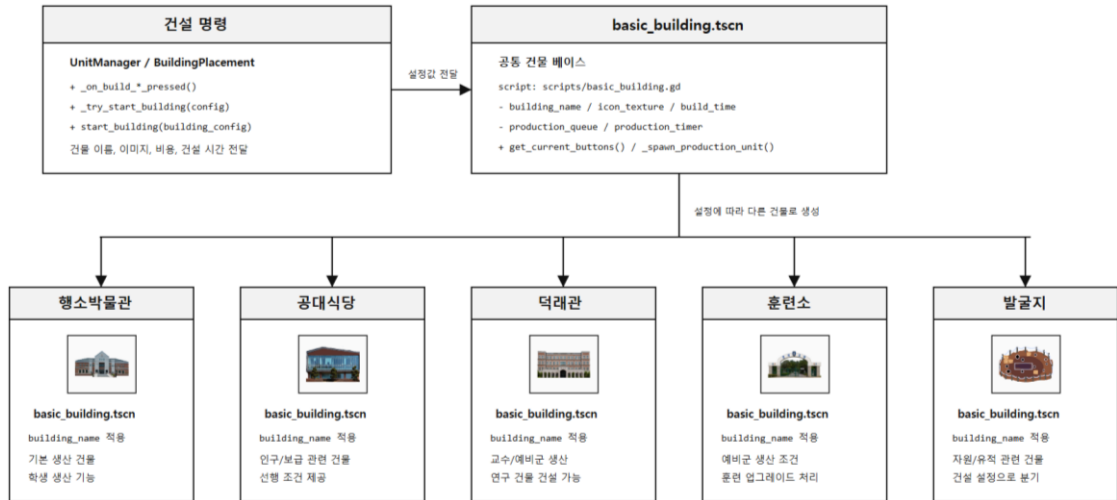


『공통된 스크립트를 사용하는 유닛들』

BasicBuilding 은 basic_building.tscn 과 basic_building.gd 를 기반으로 구현된 공통 건물 시스템이다. 건설 명령이 실행되면 건물 이름, 이미지, 건설 비용, 건설 시간, 건물 크기, 생산 가능 여부 등의 정보가 building_config 형태로 전달되며, 이를 통해 하나의 기본 구조를 다양한 건물에 적용할 수 있도록 설계하였다.

BasicBuilding 은 전달받은 설정값을 바탕으로 행소박물관, 공대식당, 덕래관, 훈련소, 발굴지 등 각 건물의 외형과 기능을 구성한다. 이를 통해 공통 구조를 유지하면서도 건물별로 생산 기능, 업그레이드 조건, 생산 가능 유닛 등의 차이를 적용할 수 있다.

또한 BasicBuilding 은 건물의 생산 기능을 담당한다. 플레이어가 생산 버튼을 선택하면 자원, 인구 수, 선행 건물 조건을 확인한 뒤 생산 대기열에 유닛을 등록한다. 건물 내부에서는 production_queue 를 이용하여 생산 대기열을 관리하고, production_timer 를 통해 생산 시간을 계산한다. 생산이 완료되면 _spawn_production_unit() 함수가 호출되어 유닛 씬이 생성되며, 생성된 유닛은 World 의 Units 노드에 추가된다.



『명령에 따라 구분되어 생성되는 건물 씬』

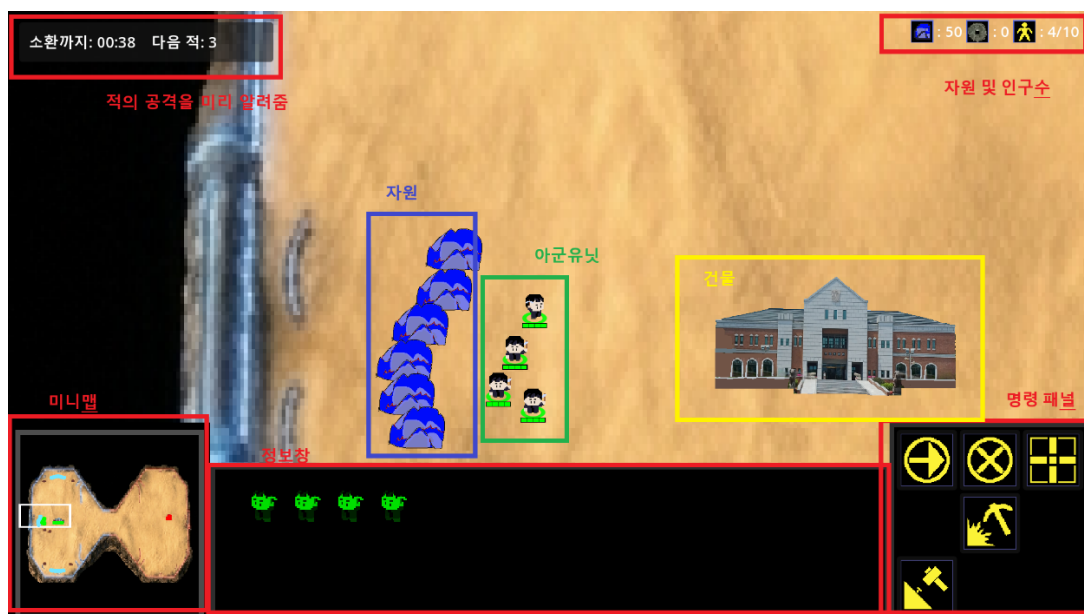
추가로 랠리 지점이 설정되어 있는 경우 `_apply_rally_order()` 함수를 통해 생성된 유닛에게 이동 또는 자원 채집 명령을 부여한다.

따라서 BasicBuilding 은 건물의 외형 표현뿐만 아니라 건설 상태 관리, 건물별 기능 적용, 유닛 생산 및 대기열 관리, 생산 완료 후 유닛 생성과 랠리 명령 처리까지 담당하는 핵심 건물 시스템이다.

전체적으로 본 시스템은 GameManager 를 중심으로 하위 요소들이 역할별로 분리된 구조를 가진다. World 는 실제 게임 대상들을 배치하고, UnitManager 는 입력과 명령을 처리하며, UI 는 플레이어에게 정보를 제공한다. Unit 과 BasicBuilding 은 각각 실제 유닛 동작과 건물 생산 기능을 수행하고, MinMap 과 Camera 는 플레이어가 전체 맵 상황을 확인할 수 있도록 보조한다.

● UI/UX 설계

UI 는 2D RTS 게임의 조작 방식에 맞추어, 플레이어가 게임 상황을 빠르게 확인하고 유닛과 건물에 명령을 내릴 수 있도록 구성하였다. 화면은 크게 게임 월드 영역과 하단 UI 영역으로 나뉜다. 게임 월드 영역에서는 유닛, 건물, 자원, 지형을 직접 확인하고 마우스 클릭이나 드래그를 통해 대상을 선택할 수 있다. 하단 UI 영역은 미니맵, 정보창, 명령 패널로 구성되어 있으며, 선택한 대상에 따라 표시되는 정보와 버튼이 달라진다.



『플레이 화면에 대한 설명』

게임 화면의 중앙 영역은 실제 플레이가 이루어지는 공간이다. 플레이어는 이 영역에서 유닛을 드래그하거나 클릭하여 선택하고, 우클릭을 통해 이동, 공격, 채집 등의 명령을 내린다. 선택된 유닛이나 건물은 시각적으로 구분되어 현재 어떤 대상을 조작하고 있는지 확인할 수 있다.

화면 하단 왼쪽에는 미니맵이 배치되어 있다. 미니맵은 전체 맵을 축소하여 보여주며, 유닛과 건물의 위치를 간단한 표시로 확인할 수 있도록 한다. 플레이어는 미니맵을 통해 현재 카메라가 보고 있는 위치와 전체 맵의 상황을 빠르게 파악할 수 있으며, 필요한 경우 미니맵을 클릭하여 카메라 위치를 이동할 수 있다.

하단 중앙에는 정보창이 배치되어 있다. 정보창은 현재 선택된 유닛이나 건물의 정보를 표시한다. 하나의 유닛을 선택하면 이름, 아이콘, 체력, 업그레이드 상태, 처치 수 등의 세부 정보가 표시되며, 여러 유닛을 선택한 경우에는 선택된 유닛들의 아이콘 목록이 표시된다. 건물을 선택한 경우에는 건물의 체력, 생산 대기열, 건설 또는 생산 진행 상태 등을 확인할 수 있다.

1) 유닛의 정보창

하나의 유닛 정보창	정보창 유닛아이콘 학생 유닛이름 0킬 킬수 체력 45/45 업그레이드 정보 체력/공격력/유닛특성
여러 유닛 정보창	정보창 유닛이 다중으로 선택된 경우 선택된 유닛을 보여주는 아이콘

2) 건물의 정보창

일반상태의 건물 정보창	건물 이미지 체력바 1000/1000 덕래관 건물의 이름 대학원생을 생산할 수 있다 건물에 대한 간단한 설명
유닛생산의 건물 정보창	인구수가 부족합니다 1000/1000 현재 생산중인 유닛이름 대학원생 다음 생산대기열 남은 시간을 보여주는 막대

하단 오른쪽에는 명령 패널이 배치되어 있다. 명령 패널은 선택된 대상이 수행할 수 있는 행동을 버튼 형태로 표시되며 버튼을 누를 수도 있고, 키보드 자판키를 사용해서 단축키로 대신 버튼을 누를 수도 있다. 유닛을 선택한 경우 이동, 공격, 정지, 채집, 건설 등의 명령 버튼이 표시되고, 건물을 선택한 경우 유닛 생산, 업그레이드와 같은 기능이 표시된다. 이를 통해 플레이어는 선택한 대상에 맞는 행동만 확인하고 실행할 수 있다.

1) 유닛생산 버튼

행소박물관		덕래관	
-------	--	-----	--

2) 업그레이드 버튼

연구실		훈련소	
-----	--	-----	--

3) 건물 선택 버튼

학생	
----	--

이러한 UI 구조는 RTS 게임에서 중요한 정보 확인과 명령 수행 과정을 한 화면 안에서 처리할 수 있도록 설계되었다. 플레이어는 월드 화면에서 상황을 확인하고, 하단 정보창에서 선택 대상의 상태를 파악한 뒤, 명령 패널을 통해 필요한 행동을 실행한다. 또한 미니맵을 통해 전체 전장을 확인할 수 있어, 유닛 조작과 전략 판단을 동시에 수행할 수 있다.

3. 테스트 및 검증

개발 중 발생한 장애요인과 문제 해결 과정

● 테스트 및 검증

직접 실행 테스트를 반복하며 주요 기능이 의도한 대로 동작하는지 검증하였다. 테스트는 유닛 선택, 이동 명령, 공격 명령, 자원 채집, 건물 건설, 유닛 생산, UI 갱신, 미니맵 표시 등 RTS 게임의 핵심 기능을 중심으로 진행하였다.

먼저 유닛 선택 기능을 검증하였다. 단일 유닛을 클릭했을 때 해당 유닛이 선택 상태로 변경되고, 정보창에 유닛 이름과 아이콘, 상태 정보가 표시되는지 확인하였다. 또한 여러 유닛을 드래그하여 선택했을 때 선택된 유닛들이 정상적으로 `unit_selected` 목록에 등록되고, 정보창에 다중 유닛 아이콘이 표시되는지 확인하였다.

이동 명령 검증에서는 선택된 유닛에게 맵의 특정 위치를 우클릭하여 이동 명령을 수행하였다. 이때 유닛이 `NavigationAgent2D` 를 통해 목적지까지 이동하는지, 여러 유닛을 동시에 선택했을 때 서로 겹치지 않고 목적지 주변으로 이동하는지 확인하였다. 장애물이 있는 구간에서는 유닛이 이동 가능한 경로를 따라 우회하는지도 테스트하였다.

건물 관련 기능은 건설 명령, 생산 대기열, 유닛 생성 과정을 중심으로 검증하였다. 학생 유닛으로 건물 건설 명령을 실행하고, 건설 가능 영역과 불가능 영역이 정상적으로 구분되는지 확인하였다. 또한 건물을 선택한 뒤 생산 버튼을 눌렀을 때 자원 조건이 확인되고, 생산할 유닛이 `production_queue` 에 등록되는지 확인하였다. 생산 시간이 완료된 후에는 `_spawn_production_unit()` 함수에 의해 유닛이 월드에 생성되는지 검증하였다.

UI 검증에서는 선택 대상에 따라 `InfoPanel` 과 `CommandCenter` 의 내용이 올바르게 변경되는지 확인하였다. 유닛을 선택했을 때는 유닛 명령 버튼이 표시되고, 건물을 선택했을 때는 생산 및 업그레이드 버튼이 표시되는지 확인하였다. 또한 자원 수치, 생산 진행 상태, 건설 진행 상태가 HUD 와 정보창에 정상적으로 반영되는지 테스트하였다.

미니맵 기능은 `SubViewport` 와 `MiniMapCamera` 를 통해 전체 맵이 축소되어 표시되는지 확인하였다. 미니맵 위에 유닛과 건물 위치가 표시되는지, 메인 카메라가 보고 있는 위치가 미니맵에서 확인 가능한지 테스트하였다. 또한 미니맵 클릭 시 메인 카메라가 해당 위치로 이동하는지도 검증하였다.

테스트 항목	검증 내용	결과
유닛 선택	- 단일 선택 - 드래그 다중 선택	정상 동작
이동 명령	- 우클릭 위치로 유닛 이동 - 다중 유닛 대형 이동	정상 동작
공격 명령	- 적 유닛 - 건물 대상 공격	정상 동작
자원 채집	- 자원 클릭 시 채집 - 반납 동작	문제 발견 후 수정/ 정상 동작
건물 건설	- 건설 가능/불가능 위치 구분 - 건설 진행	문제 발견 후 수정/ 정상 동작
유닛 생산	- 생산 버튼 클릭 - 생산 큐 등록 - 유닛 생성	문제 발견 후 수정/ 정상 동작
UI 갱신	- 선택 대상에 따른 정보창/명령 버튼 변경	정상 동작
미니맵	- 전체 맵 표시 - 유닛/건물 위치 표시 - 카메라 이동 확인	정상 동작

최종적으로 게임을 여러 차례 실행하여 기본 플레이 흐름이 정상적으로 이어지는지 확인하였다. 플레이어가 유닛을 선택하고, 자원을 채집하며, 건물을 건설하고, 유닛을 생산한 뒤 적과 전투하는 전체 흐름을 반복 테스트하였다. 테스트 과정에서 발견한 문제점은 수정한 뒤 다시 확인했으며, 최종적으로 만족스러운 결과가 나올 때까지 수정을 반복하였다. 이를 통해 각 기능이 개별적으로 동작할 뿐만 아니라 하나의 RTS 게임 플레이 흐름 안에서 서로 연결되어 동작하는지 검증하였다. 다음 항목에서는 문제 해결 과정을 서술하였다.

● 개발 중 발생한 장애요인과 문제 해결 과정

1) 자원 채집

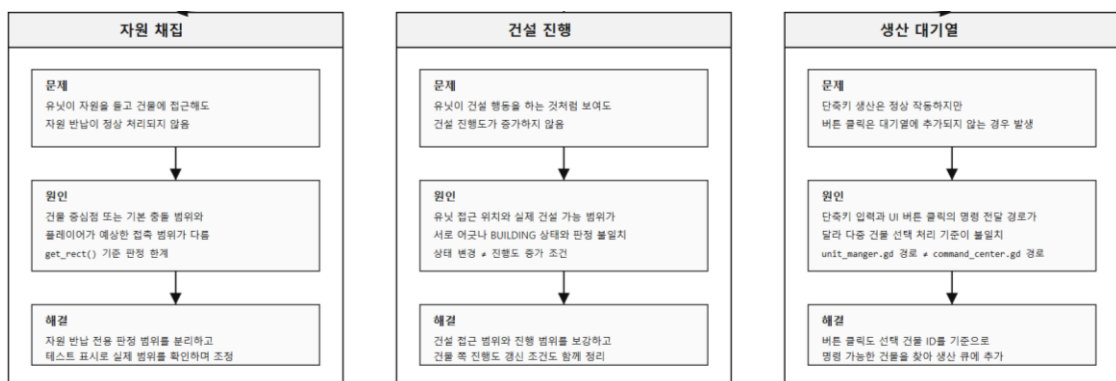
- 문제점
 - 유닛이 자원을 들고 건물에 접근해도 자원 반납이 정상 처리되지 않음
- 원인
 - 건물 중심점 또는 기본 충돌 범위와 예상한 접촉 범위가 다름
 - get_rect() 기준 판정 한계
- 해결
 - 자원 반납 전용 판정 범위를 분리
 - 테스트 표시로 실제 범위를 확인하며 조정

2) 건설 진행

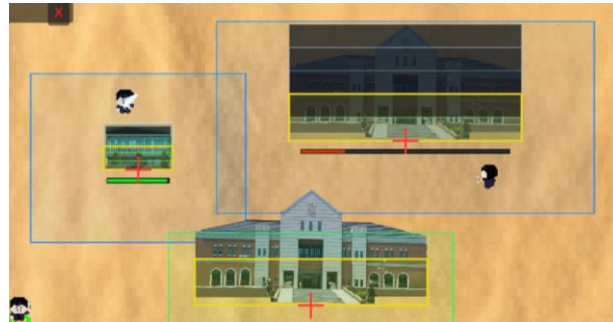
- 문제점
 - 유닛이 건설 행동을 하는 것처럼 보여도 건물 진행도가 증가하지 않음
- 원인
 - 유닛 접근 위치와 실제 건설 가능 범위가 서로 어긋나 BUILDING 상태와 판정 불일치
 - 상태 변경과 진행도 증가 조건이 다름
- 해결
 - 건설 접근 범위와 진행 범위를 보강
 - 건물 진행도 갱신 조건을 함께 정리

3) 생산 대기열

- 문제점
 - 단축키 생산은 정상 작동하지만 버튼 클릭은 대기열에 추가되지 않는 경우 발생
- 원인
 - 단축키 입력과 UI 버튼 클릭의 명령 전달 경로가 불일치
 - Unit_manger.gd 경로와 command_center.gd 경로가 불일치
- 해결
 - 버튼 클릭도 선택 건물 ID를 기준으로 명령 가능한 건물을 찾아 생산 큐에 추가



자원 채집 문제는 유닛이 자원을 들고 건물에 접근했는데도 자원 저장이 정상적으로 처리되지 않는 문제였다. 내부적으로 유닛은 건물의 중심점이나 기본 `get_rect()` 범위를 기준으로 반납 가능 여부를 판단하고 있었는데, 이 판정 범위가 실제 건물 이미지나 플레이어가 예상하는 접촉 범위와 달랐다. 이에 따라 건물의 일반 충돌 판정과 자원 반납 판정을 분리했다. 이후 자원 반납 전용 범위를 따로 계산하도록 구현하고, 유닛이 그 범위를 기준으로 접근 위치와 반납 가능 여부를 판단하게 수정했다. 또한 테스트 모드에서 판정 범위를 색상으로 표시해, 실제 인식 범위가 어디에 있는지 시각적으로 확인하면서 조정할 수 있도록 했다.



『테스트 모드에서 볼 수 있는 건물 판정 범위』

건설 과정에서는 유닛이 건설 위치에 도착해 건설 행동을 하는 것처럼 보이지만, 건설 진행도가 증가하지 않아 건물이 완성되지 않는 문제가 있었다. 내부적으로는 유닛의 상태가 `BUILDING` 상태로 바뀌었더라도, 실제 진행도 증가는 “유닛이 건설 가능 범위 안에 있는지”를 추가로 검사한 뒤 이루어졌다. 그런데 건물의 판정 범위와 유닛의 접근 위치가 어긋나면서, 유닛은 건설 행동을 하고 있지만 시스템은 아직 건설 범위 안에 들어오지 않았다고 판단하는 상황이 생겼다. 이를 해결하기 위해 건설 접근 범위와 실제 건설 진행 범위를 넓혔고, 건물이 아직 완성되기 전에도 동일한 기준으로 건설 판정을 계산하도록 수정했다. 또한 건물 쪽에서도 배정된 건설자가 있으면 진행도를 갱신할 수 있도록 보강해, 유닛과 건물 양쪽에서 진행 조건이 안정적으로 맞도록 했다.

유닛 생산 대기열 문제는 입력 처리 구조가 달라서 발생했다. 단축키 입력은 `unit_manger.gd` 에서 직접 처리되며, 현재 선택된 건물 목록을 기준으로 생산 가능한 건물을 찾아 명령을 전달했다. 반면 마우스로 버튼을 클릭하는 경우에는 `command_center.gd` 의 UI 버튼이 먼저 클릭을 받고, 버튼에 저장된 함수 이름을 통해 명령을 전달했다. 이때 버튼 클릭 경로는 단축키 경로처럼 다중 선택된 건물 중 실제로 명령을 받을 수 있는 건물을 고르는 과정이 부족했다. 그래서 어떤 건물이 업그레이드 중이거나 생산 명령을 받을 수 없는 상태이면, 단축키는 정상 작동하지만 버튼 클릭은 대기열에 유닛을 추가하지 못하는 차이가 생겼다. 해결 과정에서는 버튼 클릭도 단축키와 비슷한 구조로 명령을 전달하도록 수정했다. 선택된 건물들의 ID 를 기준으로 명령 가능한 건물을 찾고, 그 건물의 생산 대기열에 유닛을 추가하도록 처리해 두 입력 방식이 같은 결과를 내도록 맞췄다.

결과적으로 이번 수정은 단순한 UI 오류 수정이 아니라, 게임 내부의 판정 구조와 입력 명령 전달 구조를 정리하는 작업이었다. 자원 반납은 전용 인식 범위를 사용하도록 분리했고, 건설은 유닛 위치와 건물 진행 조건이 안정적으로 맞도록 범위와 상태 갱신을 보강했으며, 생산 버튼은 단축키와 동일한 명령 처리 흐름을 타도록 수정했다. 이를 통해 플레이어가 기대하는 조작 결과와 실제 내부 처리 결과가 일치하도록 개선했다.

4. 성과 및 결론

● 기대효과 및 활용방안

본 프로젝트는 2D RTS 게임 구조를 직접 구현하면서 유닛, 건물, 자원, UI, 카메라 등 다양한 오브젝트가 서로 상호작용하는 시스템을 설계하고 구현한 결과물이다. 이를 통해 RTS 게임에서 필요한 유닛 선택, 명령 전달, 건물 생산, 자원 채집, 미니맵 표시 등의 핵심 구조를 학습 예제로 활용할 수 있다.

특히 UnitManager 를 중심으로 플레이어 입력, 선택 대상 관리, 명령 전달, UI 갱신이 연결되는 구조는 다른 게임 프로젝트에서도 활용 가능한 방식이다. 이러한 중앙 관리자 구조를 통해 여러 오브젝트의 상태를 한 곳에서 관리하고, 각 오브젝트가 필요한 기능을 수행하도록 연결하는 방법을 익힐 수 있다. 또한 Godot 의 NavigationAgent2D, TileMapLayer, SubViewport, Camera2D 등을 활용하였기 때문에 Godot 기반 2D 전략 게임 개발 학습 자료로도 활용할 수 있다. 향후에는 유닛 종류, 건물 종류, 업그레이드, 적 AI, 스테이지 시스템 등을 추가하여 더 완성도 높은 RTS 게임으로 확장할 수 있다.

● 프로젝트 성과

본 프로젝트를 통해 스타크래프트와 유사한 방식의 유닛 명령 체계와 건물 생산 시스템을 구현하였다. 플레이어는 유닛을 선택하고 이동, 공격, 채집, 건설 명령을 내릴 수 있으며, 건물을 선택하면 생산 버튼을 통해 유닛을 생산 대기열에 등록할 수 있다.

유닛은 각 유닛별 씬을 따로 구성하되, 이동, 공격, 채집, 명령 처리 등 공통 동작은 unit.gd 에서 처리하도록 설계하였다. 이를 통해 유닛마다 개별적인 외형과 능력치를 가지면서도 기본 동작은 하나의 공통 구조로 관리할 수 있었다.

건물은 basic_building.tscn 과 basic_building.gd 를 기반으로 구성하였다. 건설 명령에서 전달되는 건물 이름, 이미지, 비용, 건설 시간 등의 설정값에 따라 하나의 기본 건물 구조가 여러 건물로 동작하도록 구현하였다. 또한 production_queue 와 production_timer 를 이용하여 유닛 생산 대기열과 생산 시간을 관리하였다.

UI 측면에서는 InfoPanel 과 CommandCenter 를 통해 선택 대상에 따라 정보창과 명령 버튼이 자동으로 변경되도록 구현하였다. 유닛을 선택하면 유닛 정보와 명령 버튼이 표시되고, 건물을 선택하면 건물 정보와 생산 버튼이 표시된다. 또한 SubViewport 와 MiniMapCamera 를 활용하여 전체 맵을 축소해서 보여주는 미니맵을 구현하였다.

개발 과정에서 자원 반납 판정, 건설 진행 판정, 생산 버튼 입력 경로 문제를 해결하면서 화면에서 보이는 동작과 내부 처리 결과가 일치하도록 개선하였다. 이를 통해 단순히 기능을 만드는 것뿐만 아니라, 여러 기능이 하나의 플레이 흐름 안에서 안정적으로 연결되도록 만드는 경험을 얻었다.

● 후기

이번 프로젝트를 진행하면서 여러 오브젝트가 서로 연결되어 동작하는 게임 구조를 직접 설계하는 경험을 할 수 있었다. 유닛, 건물, 자원, UI 가 각각 독립적으로 존재하는 것이 아니라, UnitManager 를 중심으로 선택, 명령, 정보 표시, 실제 동작이 연결되는 구조를 구현하면서 중앙 관리자 방식의 필요성을 느꼈다.

특히 유닛 생산과 명령 체계를 스타크래프트와 유사한 방식으로 구현할 수 있었던 점이 만족스러웠다. 유닛을 선택하고 명령 버튼을 통해 행동을 지정하거나, 건물을 선택해 생산 대기열에 유닛을 등록하는 흐름이 실제 RTS 게임처럼 동작하는 것을 확인하면서 프로젝트의 목표를 어느 정도 달성했다고 느꼈다.

또한 이번 프로젝트에서는 AI 도구의 발전을 크게 체감하였다. 처음에는 AI 없이 직접 구현을 시도했지만, 이후 AI 의 도움을 받아 코드 작성과 구조 정리를 진행하면서 개발 속도와 효율이 크게 향상되는 것을 경험하였다. 기존에 직접 작성했던 부분보다 더 효율적인 구조로 다시 만들어지는 모습을 보며 놀라움을 느꼈고, AI 가 단순한 보조 도구가 아니라 실제 개발 효율을 높이는 도구라는 점을 실감하였다.

반면 Codex 가 오류로 정상 작동하지 않았을 때는 개발 진행이 어려워지는 경험도 있었다. 이를 통해 AI 에 많은 부분을 의존하고 있다는 점을 느꼈고, AI 를 활용하더라도 개발자가 전체 구조를 이해하고 검토할 수 있어야 한다는 점이 중요하다고 생각하였다.

마지막으로 게임 개발에서는 코드뿐만 아니라 에셋 제작에도 많은 시간이 필요하다는 것을 알게 되었다. 유닛, 건물, 아이콘, UI 요소 등 필요한 시각 자료를 준비하고 게임에 맞게 적용하는 과정이 예상보다 오래 걸렸다. 이번 프로젝트를 통해 게임 개발은 기능 구현뿐만 아니라 구조 설계, 사용자 경험, 에셋 구성까지 함께 고려해야 하는 작업이라는 점을 배울 수 있었다.

5. 참고문헌

● 참고자료 및 에셋 다운로드 링크

- 게임 스타크래프트의 유닛과 참고 자료
 - 나무위키 스타크래프트 문서
<https://namu.wiki/w/스타크래프트>
 - 스타크래프트 BGM 및 음성 효과음
<https://darumju.tistory.com/entry/스타크래프트-효과음-모음>
- 고도 엔진 개발 튜토리얼 영상
 - 타일을 사용한 길 찾기 네비게이션
https://youtu.be/uS8pBQWMm5c?si=_HZYaW5rYDCjSqWS
 - 유닛 컨트롤과 유닛 그룹화 시스템
<https://youtu.be/eQjBLFKamms?si=yLcUSF9OpI1q2D2r>
- 기타 자료
 - 제작한 유닛 스프라이트의 기본 스프라이트
<https://kr.pinterest.com/pin/7529524373405024/>

『해당 문서는 맞춤법 검사, 문장 정리, 프로젝트 구성도 이미지 생성에 AI 의 도움을 받았습니다.

프로젝트 내 코드와 노드 구성에는 AI 에이전트 Codex 와 Gemini 의 도움을 받았습니다.
해당 과제는 AI 사용에 제한이 없음을 고지받았습니다.』